Autocorrect

Individual Hackathon Problem

Format	Individual
Input	One word per line on standard input
Output	One corrected word per line on standard output
Scoring	Up to 100 points for accuracy, up to 20 for speed
Limits	60 seconds wall-clock, 4 GB memory

1 Background

Autocorrect on phones and in browsers fixes most slips of the finger: `teh` becomes `the` and `recieve` becomes `receive`. It is less reliable when a writer spells a word by its sound, or does not know how the word is spelled. Each word in Table 1 is a misspelling that common autocorrectors often leave unfixed or replace with the wrong word.

Typed	Intended	Typed	Intended
buritto	burrito	gnocci	gnocchi
beaurocrat	bureaucrat	jalepeno	jalapeno
restaraunt	restaurant	cappucino	cappuccino
entreprenuer	entrepreneur	mischievious	mischievous
akwire	acquire	pharoah	pharaoh
nitch	niche	camoflage	camouflage
yot	yacht	albequerque	albuquerque
expresso	espresso	liason	liaison

Table 1: Misspellings that standard autocorrect often misses.

Your task is to build an autocorrector that fixes cases like these and still handles ordinary typos, using algorithms and data structures of your own design.

2 Problem definition

Let $\Sigma = \{\texttt{a}, \texttt{b}, \ldots, \texttt{z}, \texttt{'}\}$ be the alphabet of lowercase ASCII letters and the apostrophe, and let Σ^+ be the set of non-empty strings over Σ.

A *test case* is a sequence of n words $w_1, w_2, \ldots, w_n \in \Sigma^+$. Each w_i has a hidden reference word $y_i \in \Sigma^+$, the word the writer intended. If w_i is spelled correctly, then $y_i = w_i$.

Your program reads $w_1, \ldots, w_n$ and writes a sequence $c_1, c_2, \ldots, c_n \in \Sigma^+$, where c_i is your single best guess for y_i.

3 Input and output

Your program is run as

```
./autocorrect < input.txt > output.txt
```

Input. UTF-8 text with exactly n lines. Line i holds the word $w_i \in \Sigma^+$ and nothing else. There is no header line.

Output. Exactly n lines. Line i holds the word $c_i \in \Sigma^+$, in lowercase, and nothing else. Leading and trailing whitespace on each line is ignored.

Example.

input.txt	*output.txt*
`buritto`	`burrito`
`beaurocrat`	`bureaucrat`
`the`	`the`
`yot`	`yacht`
`recieve`	`receive`

The third word is spelled correctly, so it is returned unchanged.

4 Scoring

Accuracy. The accuracy of a run is

$$A \;=\; \frac{1}{n}\sum_{i=1}^{n} \mathbf{1}[\, c_i = y_i \,]\,,$$

where $\mathbf{1}[\cdot]$ is 1 if the condition holds and 0 otherwise. A match is an exact string comparison.

Time. The running time T is the wall-clock time from launch of your program to its exit, in seconds. It includes start-up and loading any data.

Points. Let $T_{\min}$ be the smallest running time among all valid runs. A valid run earns

$$P \;=\; \underbrace{100 \cdot A}_{\text{accuracy points}} \;+\; \underbrace{20 \cdot \frac{T_{\min}}{T}}_{\text{speed points}}\,,$$

so accuracy is worth up to 100 points and speed up to 20. The fastest valid run gets the full 20 speed points; a run twice as slow gets 10. Participants are ranked by P, highest first.

Invalid runs. A run scores $P = 0$ if any of the following hold:

- the output does not have exactly n lines;
- the program exits with a non-zero status;
- $T > 60$ seconds;
- the program uses more than 4 GB of memory.

5 Test data

- The hidden test set has 12,000 words.
- It mixes simple typos, misspellings like those in Table 1, and words that are already spelled correctly.
- Changing a correctly spelled word counts as an error, so overcorrection lowers A.
- A public development set in the same format is provided. The hidden test set does not share words with it.

6 Rules

- This is an individual competition. Each participant submits their own work.
- You may use any programming language, library, dictionary, data structure, or dataset.
- **No machine learning.** Your solution must be algorithmic. Do not use trained models of any kind, and do not call language models or other model-based services, whether locally or over the network. Dictionaries, word lists, word frequency counts, and hand-written rules (such as phonetic encodings or keyboard layouts) are allowed.
- Cite any external data or libraries in your README.
- **You must be able to explain your code.** After submission, judges may ask you to walk through your solution and answer questions about how it works. If you cannot explain a part of it, the judges may disqualify the submission.
- **No fitting to the benchmark.** Your autocorrector must be a general-purpose method for English words. Do not hard-code the words of the development set, the examples in this document, or any guessed test words, and do not add rules that exist only to fix specific test cases. Judges may run your program on fresh words outside both data sets, and a large gap between that score and your test score is grounds for disqualification.

7 Submission

Submit a single archive containing:

1. your source code;
2. a build script, or a prebuilt executable named `autocorrect` that runs on Linux x86-64;
3. a README describing your approach and how to build and run it;
4. your accuracy A and running time T on the public development set.